FAILURE DETECTIVE AI

Updated Development Schema | Hosted AI + Rule-Based Diagnostics + Simulated Sensor Classification

1. Project Overview

Failure Detective AI is a browser-accessible mechanical fault diagnosis prototype built with Python and Streamlit. It combines a hosted conversational AI assistant with a transparent rule-based diagnostic engine. Users can describe machine symptoms, enter simulated sensor readings, record inspection findings, rank possible faults, and export a diagnostic report.

2. Objectives

• Provide accessible browser-based mechanical troubleshooting without requiring users to install a local language model.
• Map user-described symptoms to a predefined mechanical fault knowledge base.
• Rank plausible fault categories using symptoms and recorded inspection evidence.
• Demonstrate device-data classification using simulated vibration, temperature, and current readings.
• Recommend follow-up inspections and export a report for presentation.

3. Updated System Architecture

Browser User Text symptoms / sensor inputs
↓
Streamlit Application UI, session state, input validation
↓
Two parallel processing paths
■ ■
Hosted AI (Groq API) Conversation and explanation
Rule-Based Diagnostic Engine Symptom matching, evidence scoring, ranking
■ ■
Diagnosis Interface Candidates, next inspection, history, downloadable report

Note: Hosted AI provides natural-language explanations. The rule-based engine is responsible for evidence scores and candidate ranking; the AI does not independently change those scores.

4. Main Modules

4.1 User Interface: Streamlit web interface with dark industrial styling, animated technical background, chat interface, sensor input panel, inspection workflow, and report download.

4.2 Hosted AI Assistant: Sends chat context and the current diagnostic evidence to a hosted Groq model using a securely stored API key in Streamlit Secrets. No local Ollama server is required for hosted responses.

4.3 Knowledge Base: Contains 18 predefined mechanical failure categories with associated symptoms and inspection checks. Keyword mapping translates common natural-language phrases into known symptom labels.

4.4 Rule-Based Reasoning: Matching symptoms add heuristic evidence points; observed inspection findings add support; findings not observed reduce the score. Scores are rankings, not probabilities.

4.5 Simulated Sensor Classification: Accepts demonstration readings such as vibration, temperature, and motor current. Configured illustrative thresholds map readings to candidate symptoms before the existing diagnostic rules rank faults.

4.6 Inspection and Reporting: Recommends uncompleted checks for supported candidates, records user-entered inspection outcomes, maintains history, and exports a text report.

5. Runtime Sequence

1. User opens the deployed Streamlit application.
2. User enters a machine description or simulated sensor values.
3. Text is mapped to known symptom labels; sensor inputs are evaluated against the demonstration thresholds.
4. The rule engine ranks candidate faults using symptoms and recorded inspection outcomes.
5. The hosted AI explains the evidence and asks clarifying questions when necessary.
6. User records an inspection finding; the app reassesses the candidates.
7. User reviews the history and downloads the diagnostic report.

6. Technology Stack

Layer	Technology / role
Application	Python, Streamlit
Hosted language model	Groq API; model configured through app settings/secrets
Secrets	Streamlit Community Cloud Secrets (GROQ_API_KEY)
Diagnostic logic	Python dictionaries, keyword mapping, heuristic evidence scoring
Device input for demonstration	Simulated/manual vibration, temperature, and current readings
Deployment	GitHub repository + Streamlit Community Cloud

7. Limitations and Safety

• Sensor values are simulated or manually entered; the prototype does not claim to read physical sensors.
• Diagnostic scores are heuristic and are not calibrated probabilities or a substitute for professional inspection.
• Results depend on the predefined knowledge base and configured demonstration thresholds.
• Hosted AI requires internet connectivity and a valid API key/model. API quotas or provider changes may affect availability.
• Users should follow site procedures and qualified personnel guidance for rotating, electrical, hot, or pressurized machinery.

8. Demonstration Plan

• Motor overheating + restricted airflow → demonstrate cooling-related candidate ranking.
• Excessive vibration + hot bearing + abnormal noise → demonstrate bearing/imbalance candidates.
• Low pump flow + high discharge pressure → demonstrate pump blockage candidate.
• Record an inspection finding and show how the ranking changes; download the report.

9. Evaluation Summary

The prototype demonstrates an end-to-end software pipeline for mechanical fault diagnosis: browser input, device-style data entry, classification, evidence-based ranking, AI-generated explanation, iterative inspection, and reporting. It is designed for educational demonstration and requires validation before use in real maintenance decisions.